SAÉ Apprentissage pour l'IA
Projet

Présentation du projet

Ce projet se découpe en deux études différentes réalisées en suivant un schéma similaire. Dans chaque étude, vous étudierez un ensemble de données spécifique et développerez différents modèles d'apprentissage profond adaptés à ces données.

Consignes générales

Ce projet se fera par groupes de 2 à 3 étudiants.

Chaque étude fera l'objet d'un travail sur un notebook jupyter spécifique. Pour chaque ensemble de données, deux documents seront à rendre :

- un **petit rapport** d'étude, au format pdf, contenant la description du problème, la description des opérations effectuées, les résultats, vos commentaires et analyses (cela revient à mettre en forme les textes et résultats issus du notebook)
- **le notebook**, si possible après avoir fait une exécution complète du notebook suite à un redémarrage du noyau Python (option "Restart & Run All")

Ces 4 fichiers (2 pour chaque étude) seront à déposer **sur Moodle pour le 27 mars 2026 au plus tard.**

Chaque notebook devra être commenté pour précisé ce qui est fait, de préférence en intégrant des cellules Markdown en plus des cellules de codes Python.

Chaque rapport devra contenir une introduction, des titres de sections, ainsi que des explications rédigées de ce qui est fait, des observations faites sur les données, des traitements effectués et des résultats obtenus.

Vous veillerez à ce que, pour chaque notebook, toutes les cellules puissent être exécutées en une seule passe sans provoquer d'erreur (si l'option "Restart & Run All" est exécutée, aucune cellule ne doit provoquer d'erreur).

Une remarque importante : l'objectif du projet n'est pas d'obtenir la meilleure performance possible, mais de démontrer une compréhension claire des concepts **vus en cours**.

Les étapes de chaque étude

Chaque étude se fera sur un notebook spécifique et suivra les étapes génériques suivantes :

1. Exploration des données

- Décrire les données
- Quels sont les tailles des ensembles de données ?
- Quelle forme ont les entrées ? Quelles caractéristiques possèdent chaque donnée d'entrée ?
- Quelle forme ont les sorties ? Quelle est le sens de ces données ?
- Faire une analyse descriptive des donnée
- Commenter brièvement le type de problème à résoudre avec ces données et la difficulté de ce problème

2. Préparation des données

- Séparer les données en deux ensembles : entraînement et test
- Normaliser les données d'entrées

3. Construction d'un modèle *baseline*

- Préparation d'un modèle MLP qui servira de base de référence pour comparer les modèles.
- Prévoir une structures de stockage adaptée à la sauvegarde des résultats et leur comparaison.
- Sauvegarder les résultats du modèle *baseline* dans la structure de stockage.
- Faire une analyse des résultats.

4. Construction de modèles *avancés*

- Développement d'un modèle adapté aux données
- Etudier un aspect des hyperparamètres du modèle
- Afficher des graphiques de résultats

5. Analyse critique des résultats

- Analyser la qualité des résultats
- Commenter les résultats
- Expliquer les difficultés rencontrées

Chaque étude comprendra des consignes supplémentaires précisées dans leur section spécifique (voir les prochaines sections).

Etude 1 : Prédiction de la qualité des vins

La première étude porte sur les données *winequality* (P. Cortez et al., 2009) constituées de deux fichiers de données décrivant les propriétés et la qualité de vins blancs et de vins rouges. Elle portera en particulier sur les données des vins blancs (winequality-white.csv). L'objectif sera de produire un réseau de neurones permettant de prédire la qualité d'un vin en fonction de ses propriétés observées.

Tous les modèles, une fois entrainés, devront être sauvegardés dans des fichiers de type .keras.

Modèle *baseline* :

Le modèle *baseline* sera un Perceptron Muti-couches (MLP) avec les caractéristiques suivantes :

- une seule couche cachée de 32 neurones utilisant tous une activation de type ReLU.
- l'apprentissage se fera avec un gradient de type SGD et un taux d'apprentissage de 0,001
- la mesure de référence utilisée pour la comparaison entre modèle : erreur absolue moyenne (mean_absolute_error / mae)

A noter que l'objectif de ce modèle est de prédire une valeur réelle. En conséquence, la couche de sortie devra avoir un seul neurone sans fonction d'activation, et la fonction de coût (loss) du réseau devra calculer l'erreur quadratique moyenne (`mean_squared_error` / `mse`)

Modèles *avancés* :

Vous développerez un nouveau MLP avec 2 couches cachées qui servira de base pour étudier différents aspects :

- le nombre de neurones dans chaque couche
- l'usage ou non de couches Dropout après les couches cachées pour limiter le sur-apprentissage

L'entraînement se fera avec méthode de gradient de type Adam et un taux d'apprentissage de 0,001.

Evaluation

Afficher un graphique comparant la mesure d'évaluation des différentes architectures au fur et à mesure des itérations (sur les ensembles d'entraînement et de test).

Etude 2 : Classement d'images sur les données CIFAR-10

La seconde étude portera sur les données CIFAR-10, disponibles depuis la bibliothèque keras :

```
from keras.datasets import cifar10
(X_train, y_train), (X_test, y_test) = cifar10.load_data()
```

Si le téléchargement de ces données pose des difficultés, veuillez le signaler rapidement afin de mettre en plus des solutions alternatives de récupération (données dans le dossier commun ou sur Moodle).

Tous les modèles, une fois entrainés, devront être sauvegardés dans des fichiers de type .keras.

Exploration des données

- Voir la documentation de la base CIFAR-10 pour avoir le sens des données de sorties
- Afficher des exemples d'images pour chaque classe
- Afficher la répartition des classes

Modèle *baseline* :

Le modèle *baseline* sera un Perceptron Muti-couches (MLP) avec les caractéristiques suivantes :

- une seule couche cachée de 32 neurones utilisant tous une activation de type ReLU.
- l'apprentissage se fera avec un gradient de type SGD et un taux d'apprentissage
- la mesure de référence utilisée pour la comparaison entre modèle : précision de classement (accuracy)

Modèles *avancés* :

Vous développerez et comparerez deux architectures de type Réseaux de neurones convolutifs (CNN) similaires à ceux testés en cours (avec deux blocs de convolutions suivis d'une couche dense). Un

réseau utilisera des filtres de taille (3x3) et un autre des filtres de taille (5x5). La méthode d'optimisation utilisera une méthode de gradient de type Adam avec un taux d'apprentissage de 0,001.

Vous étudierez l'intérêt de l'usage de l'augmentation de données (Data Augmentation).

Evaluation et analyse

Vous produirez les graphiques suivants :

- un graphique comparant la précision des modèles au fur-et-à-mesure des itérations de l'entraînement (sur les ensembles d'entraînement et de test).

- un graphique comparant le coût des modèles au fur-et-à-mesure des itérations de l'entraînement (sur les ensembles d'entraînement et de test).

L'analyse devra comprendre les éléments suivants :

- Analyser les erreurs les plus fréquentes.
- Montrer quelques exemples d'images mal classées

Bonus

Faire une analyse des confusions entre chat et chien.

Conseils sur la sauvegarde des résultats

Les entraînements des modèles peuvent prendre du temps. Si vous ne sauvegardez pas vos résultats, vous serez obligé de relancer chaque entraînement à chaque fois que vous rouvrez un notebook, ce qui n'est pas toujours réalisable dans un temps raisonnable. Il est préférable de sauver vos résultats pour pouvoir les recharger plus tard ou pour pouvoir utiliser les images dans les rapports.

Comment enregistrer les images produites par un notebook ?

Pour récupérer les images, une solution simple consiste à faire un export du notebook au format html (Menu « File », puis l'option « Save and Export Notebook AS / HTML »). Cela vous permettra de télécharger une version html de votre notebook. Si vous ouvrez ce fichier htlml dans un navigateur, vous pourrez enregistrer chaque image en faisant un clic droit sur une image et en sélectionnant « Enregistrer l'image sous » (ou une option équivalente selon le navigateur).

Comment sauvegarder des données en Python (par exemple, votre dictionnaire de résultat) ?

Python permet d'enregistrer le contenu de n'importe quelle variable grâce à un format de données appelé « pickle » qui a l'avantage de conserver l'organisation interne des données dans la variable.

```
import pickle

# Sauvegarde du contenu de « res » dans un fichier nommé « data_res.pickle »
f = open("data_res.pickle", "wb")     # Attention le "b" est important
pickle.dump(res, f)
f.close()


# Chargement des données du fichier « data_res.pickle » dans une variable « res »
f = open("data_res.pickle ", "rb")     # Attention le "b" est important
res = pickle.load(f)
f.close()
```